

Bonus 1: Interpretare Dati HTTP e DNS per Isolare l'Attore della Minaccia

Corso di Cybersecurity – Epicode | Akatsuki: Nicola Madonna, Lorenzo Rizzuti,
Donato Barbetta, Alex Tarantino, Leonardo Onori | 7 settembre 2026

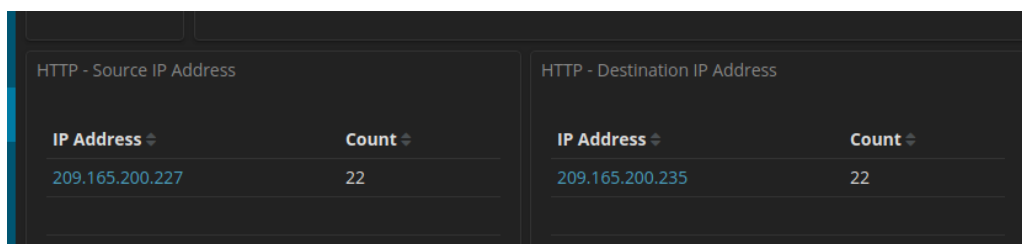
Parte 1 – Investigare un Attacco di SQL Injection

D1: Qual è l'indirizzo IP sorgente?

R: 209.165.200.227 (22 occorrenze) – dal pannello *HTTP - Source IP Address*, visibile dopo aver applicato il filtro HTTP.

D2: Qual è l'indirizzo IP destinazione?

R: 209.165.200.235 (22 occorrenze) – dal pannello *HTTP - Destination IP Address*.



The image shows two side-by-side panels from a network analysis tool. The left panel is titled 'HTTP - Source IP Address' and the right panel is titled 'HTTP - Destination IP Address'. Both panels contain a table with two columns: 'IP Address' and 'Count'. In the left panel, the IP address '209.165.200.227' is listed with a count of '22'. In the right panel, the IP address '209.165.200.235' is listed with a count of '22'.

HTTP - Source IP Address		HTTP - Destination IP Address	
IP Address	Count	IP Address	Count
209.165.200.227	22	209.165.200.235	22

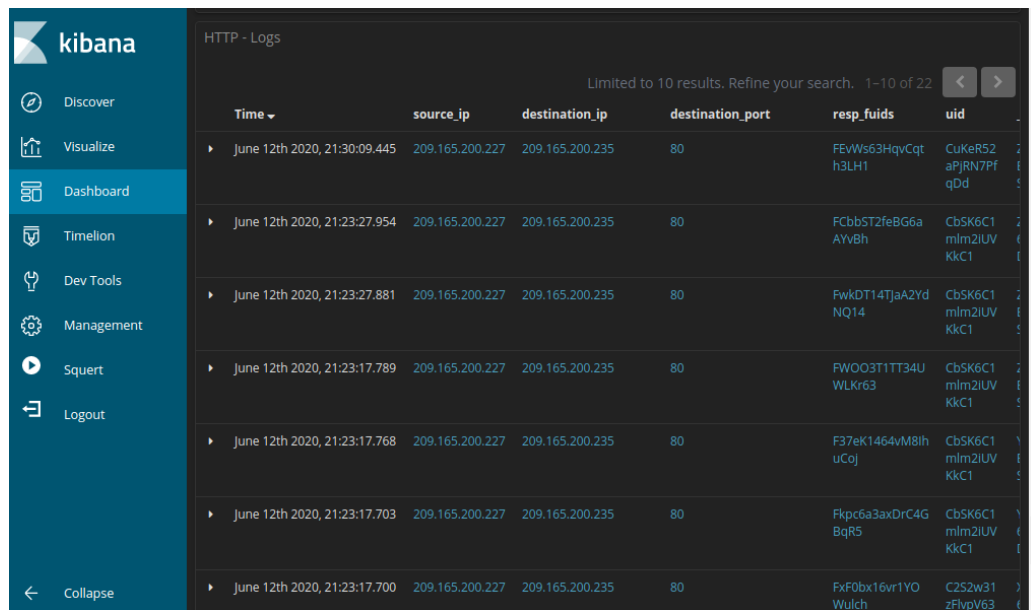
Figura 1: Pannelli HTTP - Source IP Address e HTTP - Destination IP Address.

D3: Qual è il numero di porta destinazione?

R: 80 – confermato dalla colonna `destination_port` della tabella HTTP - Logs e dal campo `id.resp_p` nel log JSON.

D4: Qual è il timestamp del primo risultato?

R: 12 giugno 2020, ore 21:30:09.445 (campo `@timestamp` del primo risultato nella vista HTTP - Logs).



Time	source_ip	destination_ip	destination_port	resp_fuids	uid
June 12th 2020, 21:30:09.445	209.165.200.227	209.165.200.235	80	FEVWs63HqvCqth3LH1	CuKeR52aPjRN7PfqDd
June 12th 2020, 21:23:27.954	209.165.200.227	209.165.200.235	80	FCbbST2feBG6aAYvBh	CbSK6C1mlm2lUVKkC1
June 12th 2020, 21:23:27.881	209.165.200.227	209.165.200.235	80	FwkDT14TjaA2YdNQ14	CbSK6C1mlm2lUVKkC1
June 12th 2020, 21:23:17.789	209.165.200.227	209.165.200.235	80	FWOO3T1TT34UWLK63	CbSK6C1mlm2lUVKkC1
June 12th 2020, 21:23:17.768	209.165.200.227	209.165.200.235	80	F37ek1464vM8lhUCoj	CbSK6C1mlm2lUVKkC1
June 12th 2020, 21:23:17.703	209.165.200.227	209.165.200.235	80	Fkpc6a3axDrC4GBqR5	CbSK6C1mlm2lUVKkC1
June 12th 2020, 21:23:17.700	209.165.200.227	209.165.200.235	80	FxF0bx16vr1YOWulch	C252w31zFlvpV63

Figura 2: HTTP - Logs, primo risultato con IP sorgente/destinazione e porta.

D5: Qual è il tipo di evento?

R: bro_http (campo event_type).

D6: Cosa è incluso nel campo message? Questi sono dettagli sulla richiesta HTTP GET fatta dal client al server. Concentrati specialmente sul campo uri nel testo del messaggio.

R: Il campo message contiene il record JSON completo del log Zeek/Bro relativo alla richiesta HTTP:

- ts: 2020-06-12T21:30:09.445030Z
- uid: CuKeR52aPjRN7PfqDd
- id.orig_h (IP sorgente): 209.165.200.227
- id.orig_p (porta sorgente): 56194
- id.resp_h (IP destinazione): 209.165.200.235
- id.resp_p (porta destinazione): 80
- method: GET
- host: 209.165.200.235
- uri: /mutillidae/index.php?page=user-info.php&username='+union+select+ccid,ccnumber,ccv,expiration,null+from+credit_cards+--+&password=&user-info-php-submit-button=View+Account+Details
- referrer: http://209.165.200.235/mutillidae/index.php?page=user-info.php
- user_agent: Mozilla/5.0 (X11; Linux x86_64; rv:68.0) Gecko/20100101 Firefox/68.0

- request_body_len / response_body_len: 0 / 23665 byte
- status_code / status_msg: 200 / OK
- tags: HTTP::URI_SQLI
- resp_fuids: FEvWs63HqvCqth3LH1
- resp_mime_types: text/html

Il campo uri in particolare mostra un tentativo di SQL injection tramite il parametro username: la query originale viene interrotta con un apice (') e sostituita con un'istruzione UNION SELECT che tenta di estrarre le colonne ccid, ccnumber, ccv, expiration dalla tabella credit_cards.



Figura 3: capME! – Log entry JSON completo con la query SQL injection nel campo uri.

D7: Qual è il significato di queste informazioni?

R: I dati confermano un attacco di SQL injection basato su UNION riuscito contro l'applicazione web Mutillidae (`index.php?page=user-info.php`), ospitata su un server Apache/2.2.8 con PHP/5.2.4. L'attaccante (209.165.200.227) ha bypassato il modulo di login iniettando codice SQL nel campo username per interrogare direttamente la tabella `credit_cards`. Il tag automatico `HTTP::URI_SQLI` generato da Zeek conferma il rilevamento dell'exploit. La risposta HTTP 200 OK con un corpo di 23.665 byte indica che il server ha effettivamente restituito dati (coerente con il messaggio "5 records found" osservato nella trascrizione capME!), a conferma che l'exfiltration è avvenuta con successo.

Parte 2 – Investigare l'Esfiltrazione di Dati DNS

D8: Cosa vedi più avanti nella trascrizione riguardo ai nomi utente? Fornisci alcuni esempi di nome utente, password e firma che sono stati esfiltrati.

R: Proseguendo nella trascrizione capME! oltre il punto in cui compare il termine **username**, si osserva la risposta del server contenente l'esito della query SQL injection: il messaggio *"Results for . 5 records found"* conferma che sono stati restituiti 5 record dalla tabella **credit_cards**, ciascuno con i campi **Username**, **Password** e **Signature**. I valori nel campo "Username" sono in realtà numeri di carta di credito. Dagli screenshot risultano visibili:

- Username: 4444111122223333 – Password: 745 – Signature: 2012-03-01
- Username: 7746536337776330 – Password: 722 – Signature: 2015-04-01
- Username: 8242325748474749 – Password: 461 – Signature: 2016-03-01
- Username: 7725653200487633 – Password: 230 – Signature: 2017-06-01
- Username: 1234567812345678 – Password: 627 – Signature: 2018-11-01

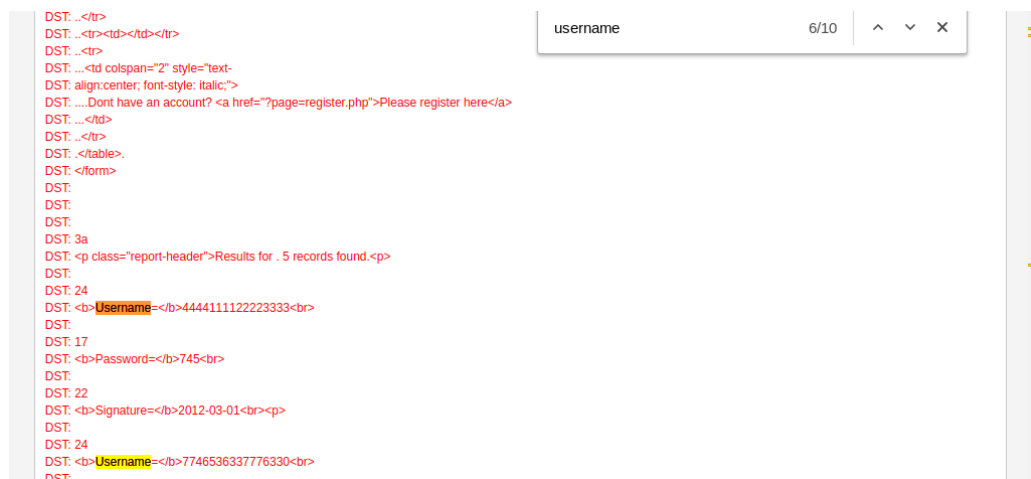


Figura 4: capME! – Primo record esfiltrato (Username, Password, Signature).



Figura 5: capME! – Record 2, 3 e 4 esfiltrati (Username, Password, Signature).

```
DST: 24
DST: <b>Username=</b>1234567812345678<br>
DST:
DST: 17
DST: <b>Password=</b>627<br>
DST:
DST: 22
DST: <b>Signature=</b>2018-11-01<br><p>
DST:
```

Figura 6: capME! – Quinto record esfiltrato, terna completa (Username, Password, Signature).

D9: Registra gli indirizzi IP del client e del server DNS.

R: Dopo aver filtrato per `example.com`: DNS Client: 192.168.0.11 (4 richieste). DNS Server: 209.165.200.235 (4 risposte).

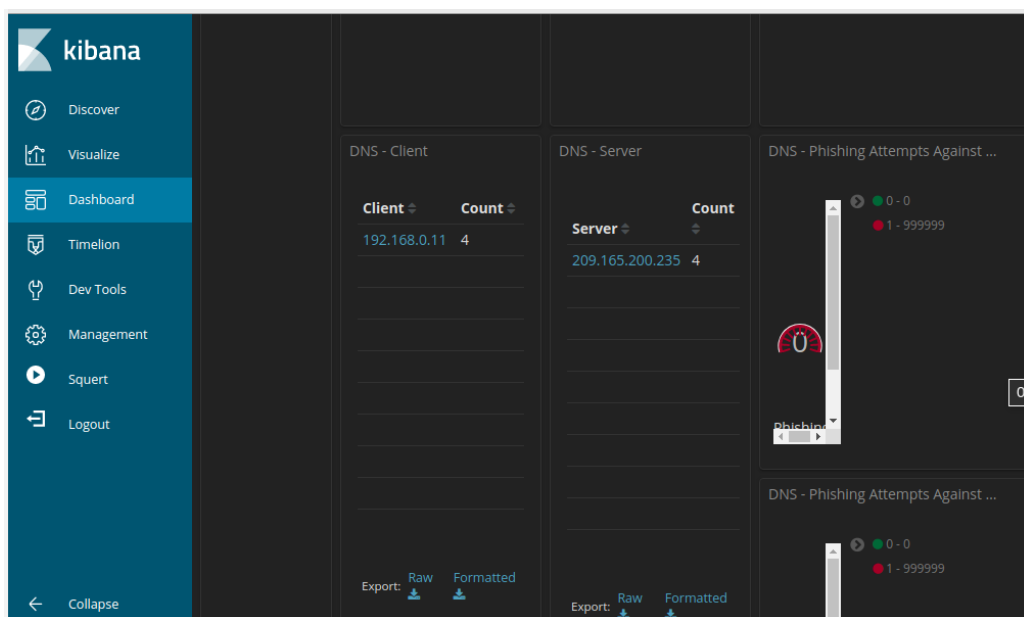


Figura 7: Pannelli DNS - Client e DNS - Server filtrati su `example.com`.

D10: I sottodomini delle query DNS erano sottodomini? Se no, qual è il testo?

R: No. Dopo aver isolato la porzione esadecimale dai 4 sottodomini di `ns.example.com` e decodificato il contenuto con `xxd -r -p "DNS - Queries.csv" > secret.txt`, il testo risultante (`cat secret.txt`) è:

```
CONFIDENTIAL DOCUMENT
DO NOT SHARE
This document contains information about the last security breach.
```

```
analyst@SecOnion:~/Downloads$ xxd -r -p "DNS - Queries.csv" > secret.txt
analyst@SecOnion:~/Downloads$ cat secret.txt
CONFIDENTIAL DOCUMENT
DO NOT SHARE
This document contains information about the last security breach.
analyst@SecOnion:~/Downloads$ █
```

Figura 8: Terminale – decodifica con xxd e visualizzazione di secret.txt.

D11: Cosa implica questo risultato riguardo a queste particolari richieste DNS? Qual è il significato più ampio?

R: Il risultato implica che le query DNS non stavano resolvendo nomi host legittimi, ma venivano usate come canale per esfiltrare dati: un documento riservato è stato codificato in esadecimale e suddiviso in etichette di sottodominio inviate come query DNS verso `ns.example.com`. Nel significato più ampio, questo dimostra che il traffico DNS – generalmente considerato innocuo e raramente ispezionato in profondità dai controlli di sicurezza perimetrali – può essere sfruttato come canale nascosto (covert channel) per far uscire dati sensibili da una rete senza destare sospetti immediati.

D12: Cosa potrebbe aver creato queste query DNS codificate e perché è stato scelto il DNS come mezzo per esfiltrare dati?

R: Query di questo tipo sono tipicamente generate da uno script o da un tool di esfiltrazione (spesso parte di un malware o di un framework post-exploitation) che codifica automaticamente i dati rubati in esadecimale e li suddivide in etichette conformi ai limiti di lunghezza di un nome DNS, per poi inviarle come richieste di risoluzione verso un dominio controllato dall'attaccante. Il DNS viene scelto come mezzo di esfiltrazione perché il traffico sulla porta 53 è quasi sempre necessario al funzionamento della rete ed è perciò consentito in uscita dalla maggior parte dei firewall, oltre a essere spesso poco monitorato o ispezionato rispetto ad altri protocolli, rendendolo un canale efficace per bypassare i controlli di sicurezza.